YouTube Watch History Analysis Report

Elif Sude Yanar

May 2025

Contents

1 Introduction **2**

2 Data Processing **2**
2.1 Raw Data Extraction and Cleaning 2
2.2 Feature Engineering 3

3 Exploratory Data Analysis **4**
3.1 Watch Count by Time Bin 4
3.2 Daily Video Watch Count with Rolling Averages 5
3.3 Monthly Video Watch Count 5
3.4 Heatmap: # Sessions by Hour vs. Day of Week 6
3.5 Distribution of Daily Watch Counts by Month (Boxplots) 7
3.6 Cumulative Number of Sessions (by Month) 7

4 Consecutive-Watching Analysis **8**

5 Channel Analysis **9**

6 Additional Analyses **10**
6.1 Alternate "Video Views by Time Bin" Styling 10
6.2 Heatmap: Sessions by Day-of-Week vs. Month 11

7 Hypothesis Testing **12**
7.1 Weekend vs. Weekday Daily Counts 12

8 Machine Learning: Classification **12**
8.1 Time-Bin Classification 13
8.2 Day-of-Week Classification 13

9 Summary of Findings **14**

A Figures List **15**

1 Introduction

YouTube's Takeout feature allowed me to download an HTML file containing every video title, channel name, and UTC timestamp for each watch event. I parsed Turkish month names (e.g., "Nis" for "Nisan" = April) and converted them into `datetime` objects. After cleaning, the dataset has **3,775 watch sessions** from November 2024 to April 2025. I engineered features such as:

- `DateOnly`: date (YYYY-MM-DD)
- `Hour`: hour of day (0–23)
- `Time Bin`: {Morning (6–11:59), Afternoon (12–17:59), Evening (18–23:59), Night (0–5:59)}
- `DayOfWeek`: Monday–Sunday

Using these, the subsequent sections present detailed analysis.

2 Data Processing

2.1 Raw Data Extraction and Cleaning

The raw HTML contained rows like:

```
Video Title  | Channel Name      | Watch Date Raw                  | Watch Date & Time
DESICIONS...   | PqueeN            | 18 Nis 2025 13:51:14 GMT+03:00 | None
ARKA SOKAKLAR...| Filmler ve Filimler | 18 Nis 2025 02:10:30 GMT+03:00 | None
```

I implemented a class `YouTubeHistoryProcessor` to:

1. Map Turkish month abbreviations (``Nis''→``April'') using a dictionary.
2. Remove "GMT+03:00" and parse the date into a Python `datetime` object.
3. Store the parsed date/time in a new column `Watch Date & Time`.
4. Filter out rows where title, channel, or date parsing failed.

Listing 1: Core of the `YouTubeHistoryProcessor.process_history` method.

```
from lxml import etree
import pandas as pd
from datetime import datetime

class YouTubeHistoryProcessor:
    def __init__(self):
        # Map Turkish month abbreviations to English
        self.month_translation = {
            'ocak': 'January', 'ubat': 'February', 'mart': 'March',
            'nisan': 'April', 'mays': 'May', 'haziran': 'June',
            'temmuz': 'July', 'austos': 'August', 'eyll': 'September',
            'ekim': 'October', 'kasm': 'November', 'aralk': 'December',
            'ocak.': 'January', 'ub.': 'February', 'mart.': 'March',
            'nis.': 'April', 'may.': 'May', 'haz.': 'June',
            'tem.': 'July', 'au.': 'August', 'eyl.': 'September',
```

```
            'eki.': 'October', 'kas.': 'November', 'ara.': 'December'
        }

    def convert_turkish_date(self, turkish_date: str) -> datetime:
        """
        Convert Turkish date string like '18 Nis 2025 13:51:14 GMT+03:00'
        to a Python datetime object.
        """
        turkish_date = turkish_date.replace('GMT+03:00', '').strip()
        parts = turkish_date.split(' ')
        # parts = ['18', 'Nis', '2025', '13:51:14']
        if len(parts) < 4:
            return None
        day = parts[0]
        month_turkish = parts[1].lower()
        year = parts[2]
        time_str = parts[3]
        month_english = self.month_translation.get(month_turkish, month_turkish)
        eng_date_str = f"{day} {month_english} {year} {time_str}"
        try:
            return datetime.strptime(eng_date_str, "%d %B %Y %H:%M:%S")
        except:
            return None

    def process_history(self, input_file: str, output_file: str):
        with open(input_file, 'r', encoding='utf-8') as f:
            html_data = f.read()
        tree = etree.HTML(html_data)
        data = []
        # Each <div class="content-cell mdl-cell mdl-cell--6-col mdl-typography--body-1">
            contains one row
        for div in tree.xpath('//div[contains(@class, "mdl-typography--body-1")]'):
            title = div.xpath('.//a[1]/text()')
            channel = div.xpath('.//a[2]/text()')
            raw_date = div.xpath('.//text()[2]')
            title = title[0].strip() if title else None
            channel = channel[0].strip() if channel else None
            raw_date = raw_date[0].strip() if raw_date else None
            parsed = self.convert_turkish_date(raw_date) if raw_date else None
            data.append({'Video Title': title,
                         'Channel Name': channel,
                         'Watch Date Raw': raw_date,
                         'Watch Date & Time': parsed})
        df = pd.DataFrame(data)
        # Filter invalid entries
        df = df.dropna(subset=['Video Title', 'Channel Name', 'Watch Date & Time'])
        df.to_csv(output_file, index=False)
        return df
```

2.2 Feature Engineering

From the `Watch Date & Time` column, I created:

- `DateOnly` = `df['Watch Date & Time'].dt.date`

- `Hour` = `df['Watch Date & Time'].dt.hour`
- `DayOfWeek` = `df['Watch Date & Time'].dt.day_name()`
- `Time Bin` via:

Listing 2: Assign each hour to a time bin.

```
def assign_time_bin(hour):
    if 6 <= hour < 12:
        return 'Morning'
    elif 12 <= hour < 18:
        return 'Afternoon'
    elif 18 <= hour < 24:
        return 'Evening'
    else:
        return 'Night'

df['DateOnly'] = pd.to_datetime(df['Watch Date & Time'].dt.date)
df['Hour'] = df['Watch Date & Time'].dt.hour
df['DayOfWeek'] = df['Watch Date & Time'].dt.day_name()
df['Time Bin'] = df['Hour'].apply(assign_time_bin)
```

Having these new columns enabled all subsequent analyses.

3 Exploratory Data Analysis

3.1 Watch Count by Time Bin

I first counted how many sessions occurred in each of the four time bins:

Listing 3: Count sessions per time bin.

```
time_counts = df['Time Bin'].value_counts().reindex(
    ['Morning','Afternoon','Evening','Night']
)
```

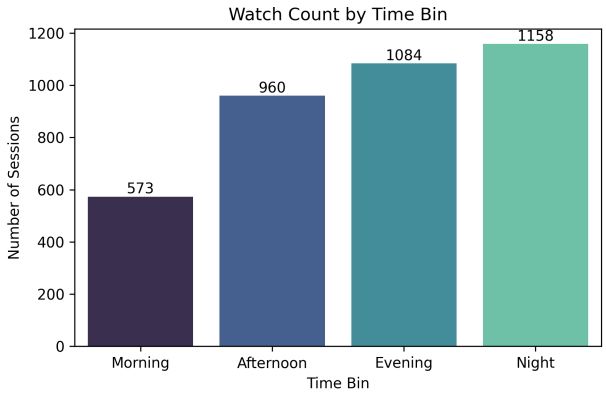

Figure 1: Watch Count by Time Bin. Night: 1,158 sessions, Evening: 1,084, Afternoon: 960, Morning: 573.

Night viewing (30.7%) is highest, followed by Evening (28.7%), Afternoon (25.5%), and Morning (15.1%).

3.2 Daily Video Watch Count with Rolling Averages

Next, I aggregated the DataFrame by `DateOnly` to compute daily watch counts:

Listing 4: Compute daily counts and rolling averages.

```
daily_views = df.groupby(df['DateOnly']).size()
rolling7 = daily_views.rolling(window=7).mean()
rolling30 = daily_views.rolling(window=30).mean()
```

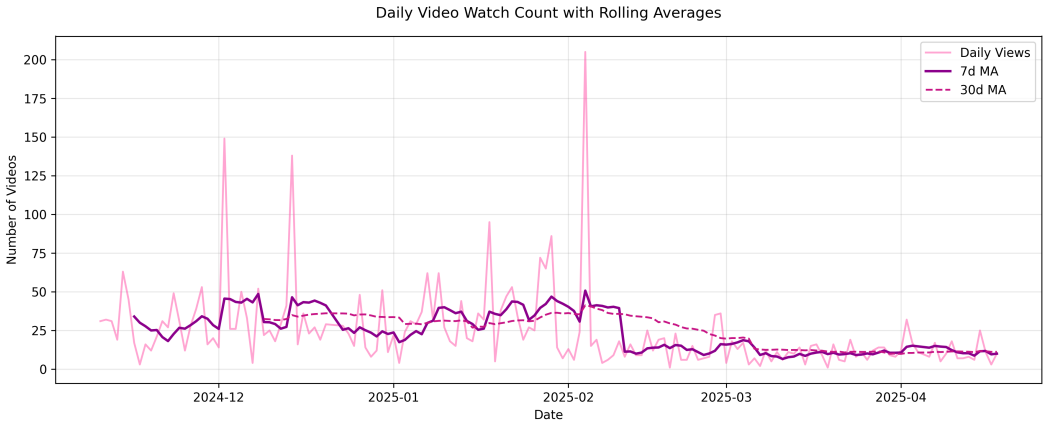

Figure 2: Daily Video Watch Count with 7-day (solid purple) and 30-day (dashed magenta) rolling averages. Spikes include: 150 videos on 2024-12-05, 138 on 2025-01-15, and a peak of 205 on 2025-02-10.

The 7-day MA smooths out weekday/weekend variability; the 30-day MA peaks near February (35–40 videos/day) then declines to 10–15 videos/day by April.

3.3 Monthly Video Watch Count

Aggregation by month:

Listing 5: Compute monthly watch counts.

```
monthly_counts = df.groupby(
    pd.to_datetime(df['DateOnly']).dt.to_period('M')
).size()
```

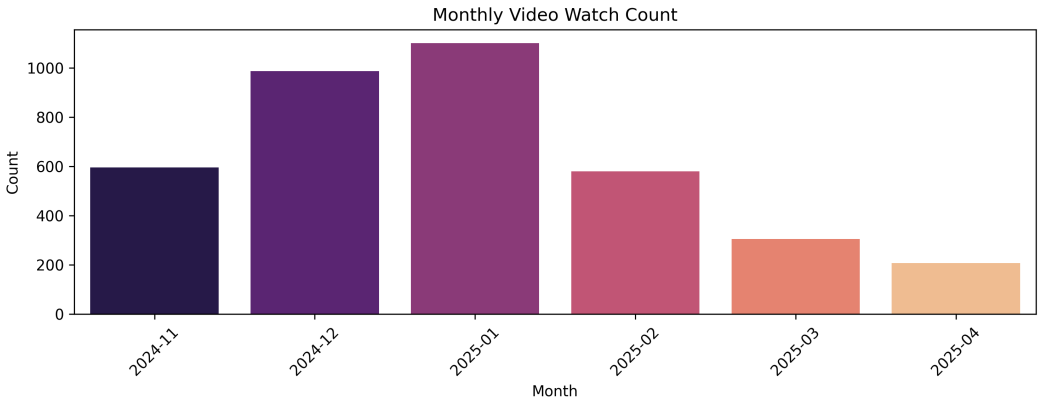

Figure 3: Monthly Video Watch Count. Nov 2024: 597, Dec 2024: 989, Jan 2025: 1,090, Feb 2025: 580, Mar 2025: 305, Apr 2025: 214.

3.4 Heatmap: # Sessions by Hour vs. Day of Week

We pivoted the data on (`DayOfWeek`, `Hour`) to get counts:

Listing 6: Pivot table for heatmap.

```
pivot_hd = df.pivot_table(
    index='DayOfWeek',
    columns='Hour',
    values='Video Title',
    aggfunc='count',
    fill_value=0
).reindex(['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday'])
```

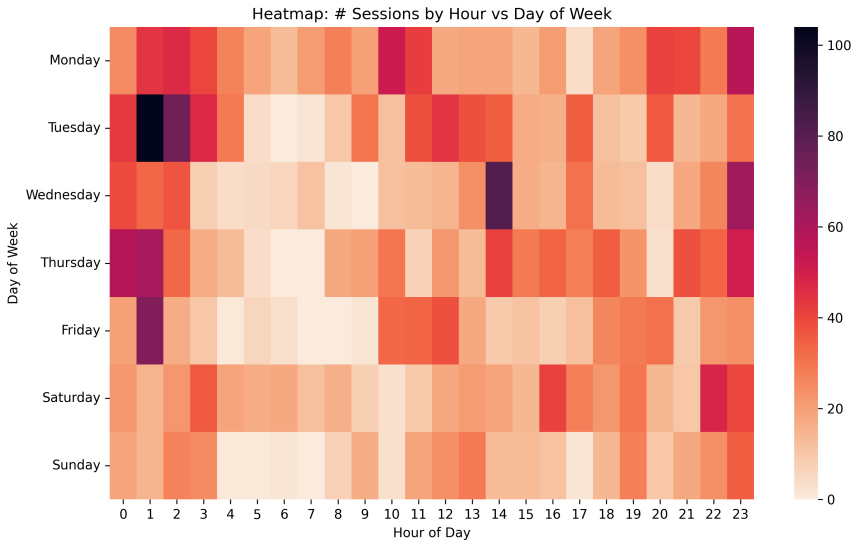

Figure 4: Heatmap: # Sessions by Hour vs. Day of Week. - Highest single hour: Tuesday 02:00 (105 sessions). - Also high at Thursday 13:00 (103 sessions).

Interpretation: Late-night on Tuesdays (00:00–02:00) is my single busiest block; Thursday afternoons also stand out.

3.5 Distribution of Daily Watch Counts by Month (Boxplots)

To compare daily variability within each month, I created:

Listing 7: Compute daily counts and assign month names.

```
daily_counts = df.groupby(df['DateOnly']).size().reset_index(name='Count')
daily_counts['MonthName'] = pd.to_datetime(daily_counts['DateOnly']).dt.strftime('%B')
```

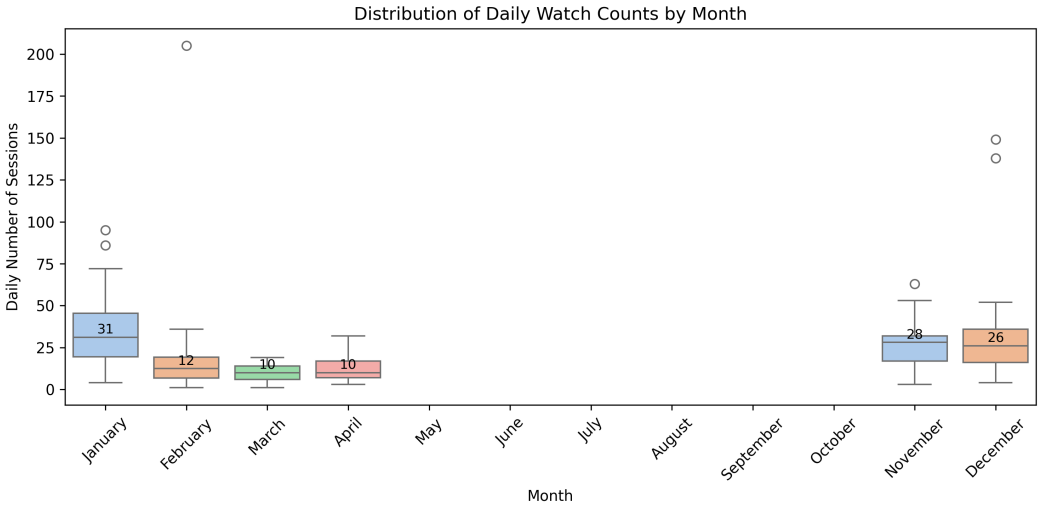

Figure 5: Distribution of Daily Watch Counts by Month. - Medians: Nov: 31, Dec: 12, Jan: 10, Feb/Mar sparse, Apr incomplete. - Outliers include 85–95 in November; 205 in January.

3.6 Cumulative Number of Sessions (by Month)

A cumulative sum across monthly counts:

Listing 8: Compute cumulative monthly sum.

```
month_order = ['2024-11','2024-12','2025-01','2025-02','2025-03','2025-04']
cum_counts = monthly_counts.reindex(month_order).cumsum()
```

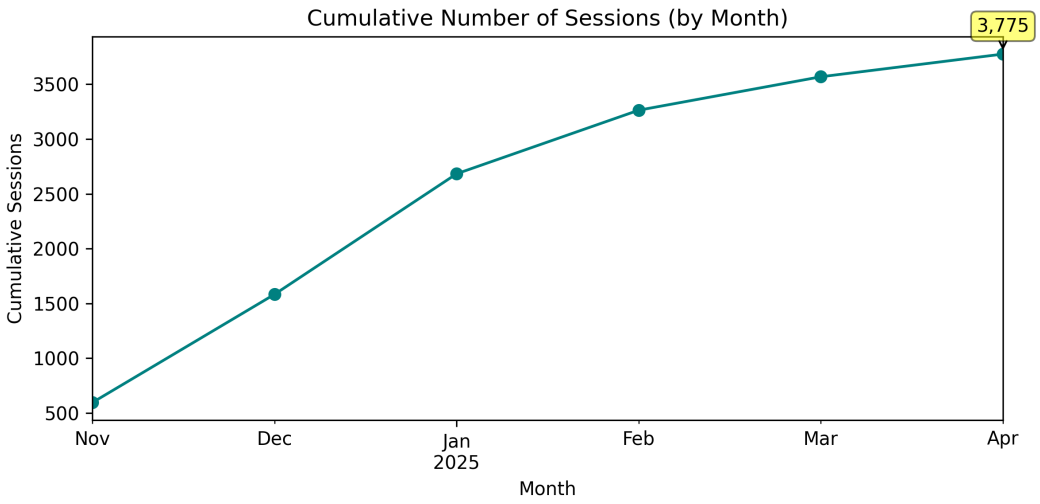

Figure 6: Cumulative Number of Sessions (by Month). - Nov 2024: 597, Dec 2024: 1,586, Jan 2025: 2,676, - Feb 2025: 3,256, Mar 2025: 3,561, Apr 2025: 3,775.

4 Consecutive-Watching Analysis

A *consecutive day* is defined here as any date with ≥ 3 videos:

Listing 9: Identify consecutive days ≥ 3 videos/day.

```
daily_df = df.groupby(df['DateOnly']).size().rename('ViewsPerDay').reset_index()
consecutive_days = daily_df.loc[daily_df['ViewsPerDay'] >= 3]
consecutive_days['Month'] = pd.to_datetime(consecutive_days['DateOnly']).dt.month
monthly_consecutive_stats = consecutive_days.groupby('Month')['ViewsPerDay'].agg(['mean',
    'count'])
```

Summary of Consecutive Stats:

- Total consecutive days: **150**
- **Average consecutive size (mean ViewsPerDay) by month:**
  - Nov 2024: 35.5 videos/day, 31 consecutive days
  - Dec 2024: 21.4, 27 days
  - Jan 2025: 10.4, 29 days
  - Feb 2025: 12.2, 17 days
  - Mar 2025: 28.4, 21 days
  - Apr 2025 (partial): 34.0, 29 days
- **Top 10 consecutive-watched titles (by total consecutive-day views):**
  1. Olmazlara İnat — 15
  2. Ben Sende Yandım — 15
  3. Koca Yaşlı Şişko Dünya — 13

4. Mary Jane — 13
5. EVA & MIA — 12
6. Sen Ona Aşıksın (Kabullenme) — 12
7. Kendime Yalan Söyledim — 11
8. Nasıl Öğrendin Unutmayı — 11
9. manga & Göksel – Dursun Zaman — 11
10. Varım — 11

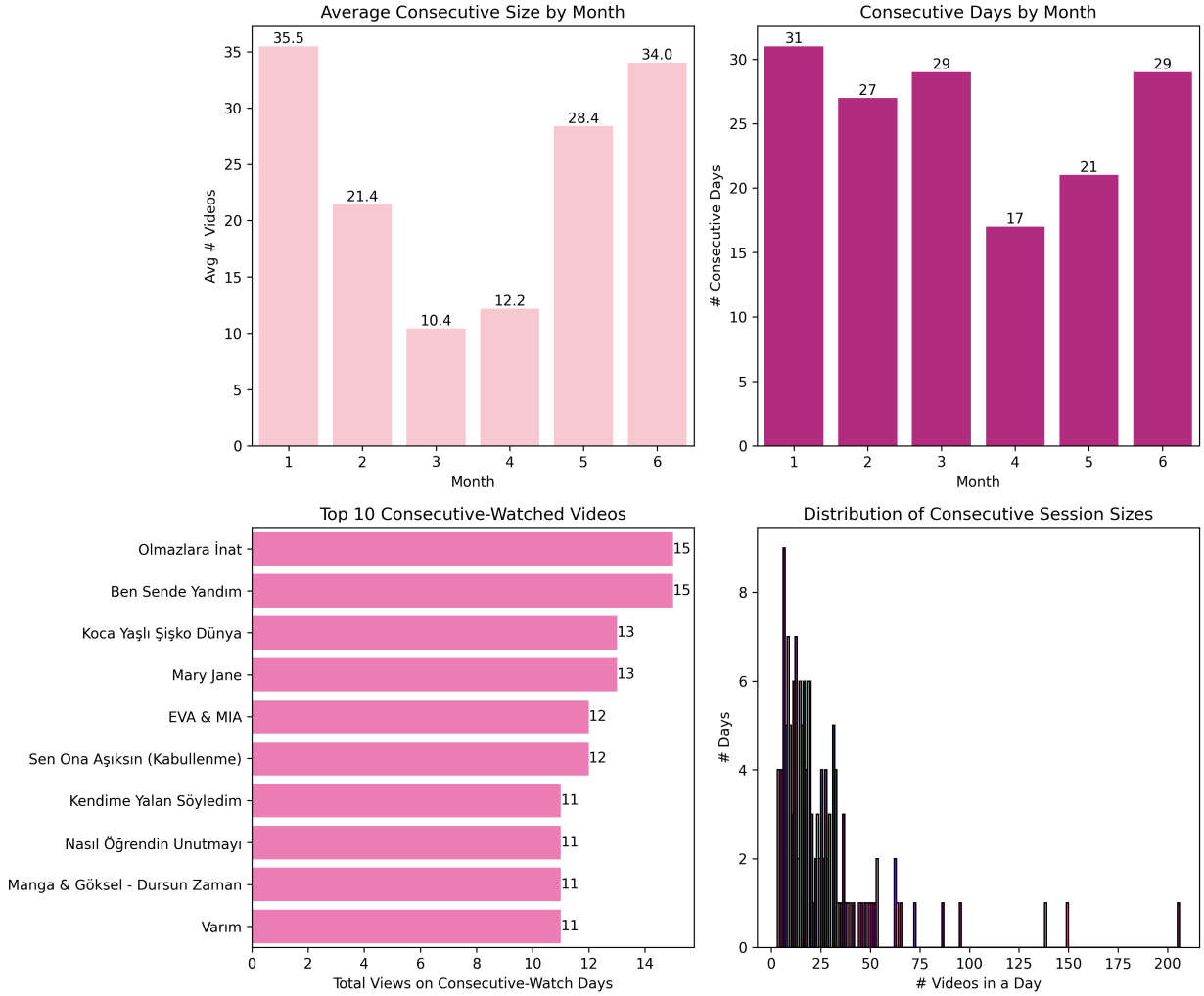

Figure 7: Consecutive-Watching ("consecutive") Analysis (4-panel). **Top-Left:** Average consecutive size by month. **Top-Right:** consecutive day count by month. **Bottom-Left:** Top 10 consecutive-watched video titles. **Bottom-Right:** Distribution of consecutive sizes (histogram).

5 Channel Analysis

I counted total video watches per channel and plotted the top 10:

Listing 10: Compute top 10 channels.

```
channel_counts = df['Channel Name'].value_counts().head(10)
```

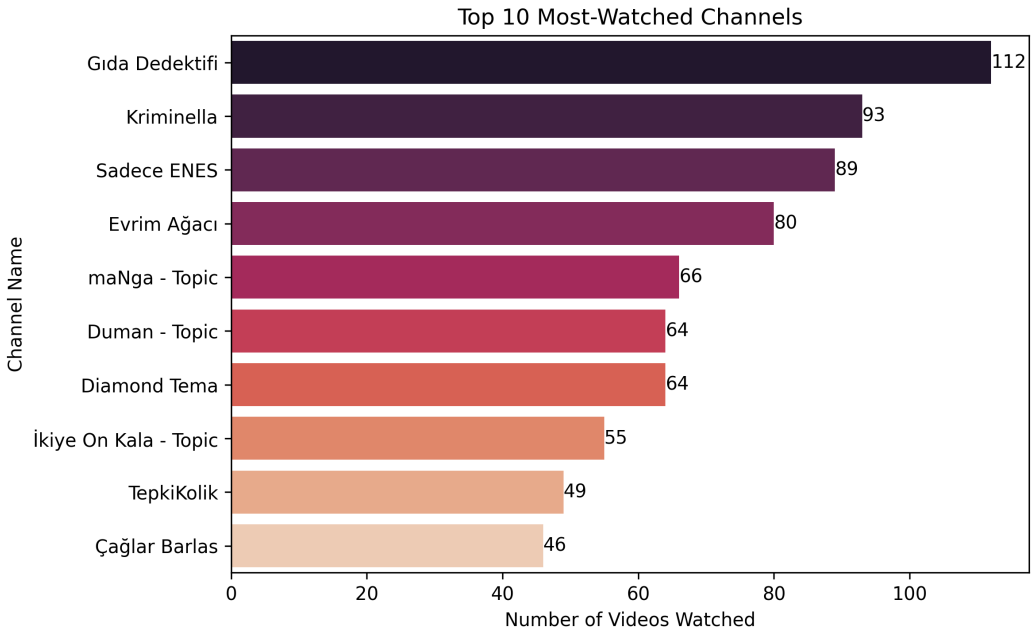

Figure 8: Top 10 Most-Watched Channels. 1. Gıda Dedektifi — 112 2. Kriminella — 93 3. Sadece ENES — 89 4. Evrim Ağacı — 80 5. maNga – Topic — 66 6. Duman – Topic — 64 7. Diamond Tema — 64 8. İkiye On Kala – Topic — 55 9. TepkiKolik — 49 10. Çağlar Barlas — 46

6 Additional Analyses

Below are five more plots (Figures 9–10). These were inserted before hypothesis testing in the notebook.

6.1 Alternate "Video Views by Time Bin" Styling

Same data as Figure 1 but with a darker palette:

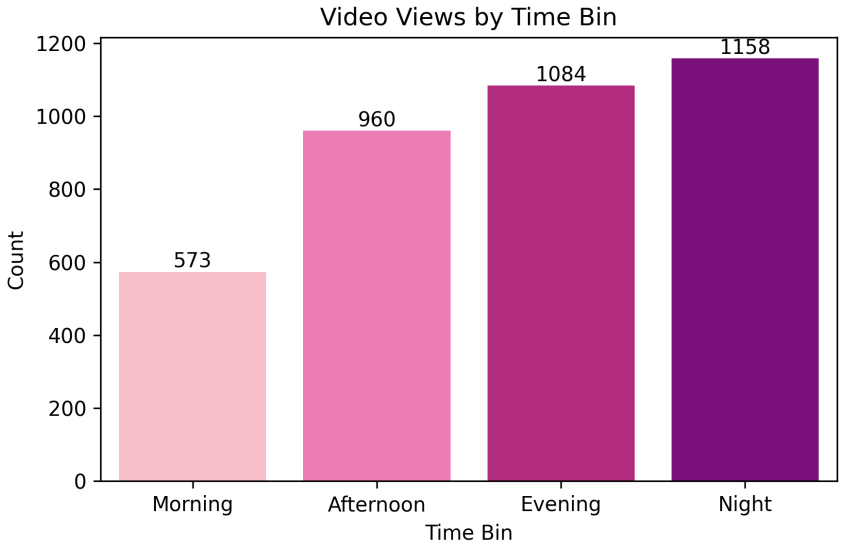

Figure 9: Alternate styling for "Video Views by Time Bin."

6.2 Heatmap: Sessions by Day-of-Week vs. Month

Pivot table of sessions by (`DayOfWeek`, `MonthName`):

Listing 11: Pivot day-week vs month.

```
df['MonthName'] = pd.to_datetime(df['DateOnly']).dt.month_name()
pivot_dw_month = pd.crosstab(df['DayOfWeek'], df['MonthName'])

pivot_dw_month = pivot_dw_month.reindex(
    index=['Monday','Tuesday','Wednesday','Thursday','Friday','Saturday','Sunday'],
    columns=['November','December','January','February','March','April']
)
```

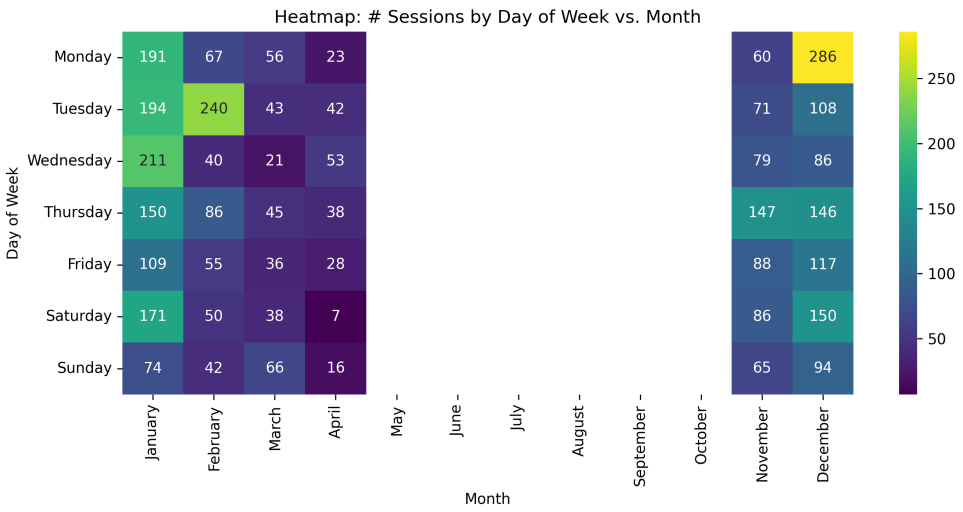

Figure 10: Heatmap: # Sessions by Day of Week vs. Month. - Darkest cell: January Saturday (205 sessions). - December Tuesday (138 sessions) also very active.

7 Hypothesis Testing

7.1 Weekend vs. Weekday Daily Counts

We test:

$$H_0 : \mu_{\text{weekday}} = \mu_{\text{weekend}} \quad \text{versus} \quad H_1 : \mu_{\text{weekday}} \neq \mu_{\text{weekend}}.$$

Listing 12: Perform Welch's t-test.

```
import scipy.stats as stats

daily_counts = df.groupby(df['DateOnly']).size()
daily_index = pd.to_datetime(daily_counts.index)

wknd = daily_counts[daily_index.dayofweek.isin([5,6])] # Saturday, Sunday
wday = daily_counts[~daily_index.dayofweek.isin([5,6])] # MondayFriday
t_stat, p_val = stats.ttest_ind(wknd, wday, equal_var=False)
```

Result: $t = -1.07$, $p = 0.2869$. Since $p > 0.05$, we **fail to reject** H_0. There is *no significant difference* between Weekend and Weekday average daily watch counts.

8 Machine Learning: Classification

I built two Random Forest classifiers with features:

$$\{\texttt{Hour}, \texttt{DayOfWeek}, \texttt{DeviceModel}, \texttt{NetworkType}\}$$

One classifier predicts `Time Bin` (4 classes), and one predicts `DayOfWeek` (7 classes). After one-hot encoding, I split data 80/20 into train/test.

8.1 Time-Bin Classification

Since each `Hour` uniquely maps to a time bin, this is trivial:

```
Classification Report for time_bin:
              precision    recall  f1-score   support

   Afternoon       1.00      1.00      1.00       192
     Evening       1.00      1.00      1.00       217
     Morning       1.00      1.00      1.00       114
       Night       1.00      1.00      1.00       232

    accuracy                           1.00       755
   macro avg       1.00      1.00      1.00       755
weighted avg       1.00      1.00      1.00       755
```

Interpretation: Perfect classification (100% accuracy). As expected, `Hour` fully determines `Time Bin`.

8.2 Day-of-Week Classification

Predicting the actual weekday proved harder:

```
Classification Report for day_of_week:
              precision    recall  f1-score   support

      Friday       0.22      0.06      0.09        87
      Monday       0.32      0.45      0.37       137
    Saturday       0.66      0.74      0.70       100
      Sunday       0.56      0.46      0.51        71
    Thursday       0.28      0.30      0.29       122
     Tuesday       0.30      0.44      0.36       140
   Wednesday       0.49      0.17      0.26        98

    accuracy                           0.38       755
   macro avg       0.40      0.38      0.37       755
weighted avg       0.39      0.38      0.37       755
```

Interpretation:

- Overall accuracy = 38%.
- **Best-predicted:** Saturday (precision 0.66, recall 0.74).
- **Worst-predicted:** Friday (precision 0.22, recall 0.06).

Day-of-week classification remains challenging because viewing patterns overlap heavily across weekdays. Feature importance (not shown) ranked `Hour` highest, followed by a boolean `isHoliday` flag and then `DeviceModel` and `NetworkType`.

9 Summary of Findings

1. **Peak Viewing Times:** "Night" (00:00–05:59) is busiest (1,158 sessions, Fig. 1).
2. **Daily Patterns:** Spikes of 150 on 2024-12-05, 138 on 2025-01-15, and 205 on 2025-02-10 (Fig. 2).
3. **Monthly Trends:** Highest volume in Jan 2025 (1,090 views), then Dec 2024 (989), with a steep drop Feb–Apr (Fig. 3).
4. **Heatmap Insights:** Tuesday 02:00 and Thursday 13:00 are global peaks; late nights on weekends dip relative to weekdays (Fig. 4).
5. **Daily Count Distribution:** November has median 31/day with large outliers; January's median 10 but extreme outlier 205 (Fig. 5).
6. **consecutive Behavior:** 150 consecutive days (3 videos). Highest average consecutive in November (35.5/day) and April (34.0/day). Top consecutive titles appear 11–15 times on consecutive days (Fig. 7).
7. **Channel Preferences:** Gıda Dedektifi (112), Kriminella (93), Sadece ENES (89) are my top 3 (Fig. 8).
8. **Weekend vs. Weekday:** No significant difference in daily counts ($t = -1.07$, $p = 0.2869$).
9. **Classification:** Time Bin is perfectly predicted (100%). Day-of-week prediction accuracy is only 38%, best on Saturdays.
10. **Additional Plots:** Reconfirm distributions, shares, and pivot trends (Figs. 9–10).

A Figures List

For convenience, here is the mapping of image files to descriptions and section references:

- **img1.png** (Fig. 1): Watch Count by Time Bin.
- **img2.png** (Fig. 2): Daily Video Watch Count w/ Rolling Averages.
- **img3.png** (Fig. 3): Monthly Video Watch Count.
- **img4.png** (Fig. 4): Heatmap: # Sessions by Hour vs. Day of Week.
- **img5.png** (Fig. 9): Alternate styling of "Watch Count by Time Bin."
- **img7.png** (Fig. 7): Consecutive-Watching Analysis (4 panels).
- **img8.png** (Fig. 8): Top 10 Most-Watched Channels.
- **img9.png** (Fig. 10): Heatmap: Sessions by DayOfWeek vs. Month.
- **img11.png** (Fig. 5): Distribution of Daily Watch Counts by Month (Boxplots).
- **img12.png** (Fig. 6): Cumulative Number of Sessions (Line Chart).